



Universidad Abierta para Adultos (UAPA)
Maestría en Big Data e Inteligencia de Negocios.
Asignatura: MBD-106- Base de Datos NoSQL

Proyecto Final: Unidad VI

**Diseño e implementación de una solución NoSQL para una plataforma
ciudadana de denuncias utilizando MongoDB**

Problema seleccionado:

#25 – Plataforma ciudadana de denuncias

Estudiante:

Jesus Daniel Montero Santana

Matricula:

100099508

Facilitador:

Dr. Alberto Verdecia Cabrera

Fecha:

Junio 2026,

Santo Domingo, Republica Dominicana



Resumen Ejecutivo

La transformación digital de las instituciones públicas ha generado la necesidad de implementar soluciones tecnológicas que permitan una comunicación más eficiente entre los ciudadanos y los organismos gubernamentales. En este contexto, el presente proyecto propone el diseño de una plataforma ciudadana de denuncias basada en tecnologías NoSQL, con el objetivo de facilitar el registro, seguimiento y gestión de incidencias reportadas por la población.

La solución está orientada a permitir que los ciudadanos puedan registrar denuncias relacionadas con problemas comunitarios como acumulación de basura, alumbrado público defectuoso, baches en las vías, contaminación ambiental y otras situaciones que afectan la calidad de vida de la comunidad. Al mismo tiempo, las autoridades competentes podrán consultar, clasificar y dar seguimiento a cada caso de manera eficiente.

Para el desarrollo de la solución se seleccionó MongoDB como motor de base de datos principal debido a su flexibilidad para manejar información semiestructurada, su facilidad de escalabilidad y su capacidad para almacenar documentos JSON de manera eficiente. Esta característica resulta especialmente útil para gestionar denuncias que pueden incluir diferentes tipos de información, evidencias fotográficas y estados de seguimiento.

La propuesta contempla el diseño de la base de datos, el análisis de los patrones de acceso, la definición de consultas clave y la arquitectura de una miniaplicación orientada a la gestión de denuncias ciudadanas. Con esta solución se espera mejorar la trazabilidad de los reportes, optimizar los tiempos de respuesta institucional y fortalecer la interacción entre los ciudadanos y las entidades gubernamentales mediante el uso de tecnologías modernas de gestión de datos.

1. Contexto y descripción de la organización

La presente propuesta se desarrolla para una organización gubernamental ficticia denominada Dirección Nacional de Atención Ciudadana (DNAC), entidad encargada de recibir, gestionar y canalizar las denuncias realizadas por los ciudadanos sobre problemas que afectan a sus comunidades.

La organización opera a nivel nacional y tiene como misión fortalecer la participación ciudadana mediante el uso de herramientas tecnológicas que permitan reportar incidencias de manera rápida, transparente y eficiente. Entre las situaciones más comunes que reciben se encuentran denuncias relacionadas con acumulación de basura, deterioro de vías públicas, alumbrado defectuoso, contaminación ambiental, daños en infraestructuras comunitarias y otras problemáticas urbanas.

Actualmente, gran parte de estas denuncias son recibidas por llamadas telefónicas, correos electrónicos o formularios dispersos, lo que dificulta el seguimiento adecuado de los casos y genera retrasos en la atención. Esta situación provoca pérdida de información, duplicidad de reportes y limitaciones para generar estadísticas que apoyen la toma de decisiones.

La Dirección Nacional de Atención Ciudadana tiene como principales actores a los ciudadanos, los operadores encargados de registrar las denuncias y las instituciones gubernamentales responsables de resolver cada incidencia. Debido al creciente volumen de información generado diariamente, la organización requiere una solución tecnológica moderna que permita almacenar y gestionar los datos de manera eficiente, garantizando disponibilidad, escalabilidad y facilidad de consulta.

Por esta razón se propone el uso de MongoDB como base de datos NoSQL principal, aprovechando su flexibilidad para manejar información diversa y su capacidad para adaptarse al crecimiento continuo de la plataforma.

2. Identificación de la necesidad

La gestión eficiente de las denuncias ciudadanas representa un desafío importante para muchas instituciones gubernamentales. Actualmente, numerosos reportes son recibidos a través de diferentes canales, como llamadas telefónicas, correos electrónicos y formularios manuales, lo que

dificulta el control centralizado de la información y el seguimiento adecuado de cada caso.

La principal necesidad identificada consiste en disponer de una plataforma digital que permita registrar, almacenar y consultar denuncias de manera organizada, facilitando la comunicación entre los ciudadanos y las entidades responsables de resolver los problemas reportados.

La ausencia de un sistema centralizado provoca retrasos en la atención, duplicidad de información, dificultades para medir el desempeño institucional y una limitada capacidad para generar estadísticas que apoyen la toma de decisiones. Como consecuencia, los ciudadanos perciben una menor eficiencia en los servicios públicos y una disminución en la confianza hacia las instituciones responsables.

La solución propuesta busca mejorar el proceso mediante una plataforma basada en MongoDB que permita almacenar denuncias de forma flexible, registrar evidencias asociadas, consultar el estado de los casos y generar información útil para la gestión institucional.

Entre los principales beneficios esperados se encuentran la reducción de los tiempos de respuesta, una mejor trazabilidad de las denuncias, mayor transparencia en el seguimiento de los casos y una mejora significativa en la calidad del servicio ofrecido a la ciudadanía.

3. Análisis de patrones de acceso

Para diseñar una solución NoSQL eficiente es necesario identificar los principales patrones de acceso que tendrá la aplicación. En la plataforma ciudadana de denuncias se identifican los siguientes procesos principales:

➤ Registro de denuncias

Los ciudadanos registran nuevas denuncias a través de la plataforma, proporcionando información como tipo de incidencia, descripción, ubicación, fecha y evidencias fotográficas.

- Tipo de carga: Transaccional.
- Frecuencia: Alta.
- Escrituras: Altas.
- Lecturas: Bajas.
- Latencia tolerable: Menor a 2 segundos.

➤ **Consulta de denuncias**

Los ciudadanos y operadores consultan denuncias previamente registradas para verificar su estado y seguimiento.

- Tipo de carga: Lectura.
- Frecuencia: Alta.
- Escrituras: Bajas.
- Lecturas: Altas.
- Latencia tolerable: Menor a 1 segundo.

➤ **Actualización de estados**

Las instituciones responsables actualizan el estado de las denuncias conforme avanza su proceso de atención.

Ejemplos de estados:

- Pendiente.
 - En proceso.
 - Resuelta.
 - Cerrada.
-
- Tipo de carga: Mixta.
 - Frecuencia: Media.
 - Lecturas y escrituras: Balanceadas.
 - Latencia tolerable: Menor a 2 segundos.

Generación de reportes y estadísticas

Los administradores requieren consultar información agregada para conocer la cantidad de denuncias por sector, categoría, fecha o estado.

- Tipo de carga: Analítica.
- Frecuencia: Media.
- Lecturas: Altas.
- Escrituras: Muy bajas.
- Latencia tolerable: Entre 2 y 5 segundos.

➤ **Justificación general**

El análisis muestra un predominio de operaciones de lectura y escritura sobre documentos semiestructurados. Debido a que las denuncias pueden contener diferentes cantidades de información, imágenes y datos asociados, MongoDB resulta una alternativa adecuada gracias a su flexibilidad, escalabilidad y facilidad para gestionar documentos JSON sin requerir esquemas rígidos.

4. Decisión de motor NoSQL

Para el desarrollo de la plataforma ciudadana de denuncias se seleccionó MongoDB como motor de base de datos principal. Esta decisión se fundamenta en las características del problema, los patrones de acceso identificados y la naturaleza de la información que será almacenada.

MongoDB permite gestionar documentos en formato BSON (Binary JSON), lo que facilita almacenar información semiestructurada sin la necesidad de definir esquemas rígidos. Esta característica resulta especialmente útil para las denuncias ciudadanas, ya que cada reporte puede contener diferentes cantidades de información, imágenes, ubicaciones y observaciones.

Además, MongoDB ofrece ventajas importantes como:

- Flexibilidad en el diseño de los documentos.
- Escalabilidad horizontal mediante sharding.
- Alto rendimiento en consultas y actualizaciones.
- Facilidad para almacenar arreglos y documentos embebidos.
- Amplio soporte y documentación en entornos empresariales.

➤ **Comparación con Cassandra**

Otra alternativa considerada fue Cassandra, una base de datos NoSQL orientada a columnas ampliamente utilizada en entornos de Big Data.

Sin embargo, Cassandra está diseñada principalmente para escenarios con volúmenes masivos de escritura distribuida y consultas previamente definidas. Aunque ofrece una gran escalabilidad y tolerancia a fallos, su modelo de datos resulta menos flexible para manejar estructuras variables como las denuncias ciudadanas.

Mientras MongoDB permite almacenar documentos completos con información heterogénea en una sola entidad, Cassandra requiere diseñar cuidadosamente las tablas según los patrones de consulta, lo que incrementa la complejidad del desarrollo para este caso.

➤ **Justificación final**

Tomando en consideración los requerimientos del proyecto, MongoDB representa la mejor alternativa debido a su flexibilidad, facilidad de implementación y capacidad para gestionar información dinámica. Estas características permiten construir una solución eficiente para el registro, seguimiento y consulta de denuncias ciudadanas, garantizando además posibilidades de crecimiento futuro de la plataforma.

5. Diseño de la base de datos

➤ **Esquema de datos**

La solución propuesta utiliza una colección principal denominada denuncias, donde se almacena toda la información relacionada con los reportes realizados por los ciudadanos.

Se optó por un modelo documental debido a que las denuncias pueden contener información variable, evidencias, comentarios y diferentes estados de seguimiento.

Ejemplo de documento JSON:

```
{
  "_id": "D001",
  "ciudadano": {
    "nombre": "Juan Pérez",
    "correo": "juanperez@gmail.com",
    "telefono": "809-555-1234"
  },
  "tipo_denuncia": "Basura",
  "descripcion": "Acumulación de basura en una vía pública.",
  "sector": "San Isidro",
  "fecha_reporte": "2026-06-15",
  "estado": "Pendiente",
}
```

```

"evidencias": [
  "foto1.jpg",
  "foto2.jpg"
],
"comentarios": [
  {
    "fecha": "2026-06-16",
    "detalle": "Caso recibido por el ayuntamiento."
  }
]
}

```

➤ **Justificación del diseño**

Se utilizaron documentos embebidos para almacenar la información del ciudadano y los comentarios relacionados con cada denuncia. Esta estrategia reduce la necesidad de realizar múltiples consultas y mejora el rendimiento de lectura.

Asimismo, se emplean arreglos para almacenar múltiples evidencias y comentarios asociados a una misma denuncia.

◆ **Tipos BSON utilizados**

Campo	Tipo BSON
_id	ObjectId
ciudadano.nombre	String
ciudadano.correo	String
ciudadano.telefono	String
Tipo_denuncia	String
descripcion	String
sector	String
fecha_reporte	Date
estado	String
evidencias	Array
comentarios	Array
Comentarios.fecha	Date
Comentario.detalle	String

La selección de estos tipos BSON permite almacenar la información de forma eficiente y compatible con las funcionalidades nativas de MongoDB.

Índices

Los índices son estructuras que permiten acelerar las consultas realizadas sobre la base de datos. Debido a que los usuarios consultarán frecuentemente las denuncias por estado, tipo de denuncia y sector, se propone la creación de los siguientes índices.

➤ Índice por estado

```
db.denuncias.createIndex({ estado: 1 })
```

Este índice facilita las consultas relacionadas con el seguimiento de denuncias pendientes, en proceso o resueltas.

➤ Índice por tipo de denuncia

```
db.denuncias.createIndex({ tipo_denuncia: 1 })
```

Permite localizar rápidamente denuncias asociadas a una categoría específica, como basura, alumbrado público o contaminación ambiental.

➤ Índice compuesto

```
db.denuncias.createIndex({  
  estado: 1,  
  sector: 1,  
  fecha_reporte: -1  
})
```

Este índice optimiza consultas donde se filtran denuncias por estado y sector, ordenándolas además por fecha de reporte.

➤ Justificación utilizando la regla ESR

La regla ESR (Equality, Sort, Range) establece que los campos utilizados para igualdad deben colocarse primero, seguidos por los campos de ordenamiento y finalmente los campos utilizados en filtros de rango.

En el índice compuesto propuesto:

- Equality (E): estado y sector.
- Sort (S): fecha_reporte.
- Range (R): no aplica en este escenario.

La aplicación de esta regla permite reducir la cantidad de documentos examinados y mejorar significativamente el rendimiento de las consultas más frecuentes de la plataforma.

◆ **Particionamiento o Sharding**

A medida que la plataforma crezca y aumente la cantidad de denuncias registradas, MongoDB ofrece la posibilidad de implementar sharding para distribuir los datos entre varios servidores.

Para este proyecto se propone utilizar el campo sector como posible clave de particionamiento, debido a que las consultas suelen realizarse por ubicación geográfica y zonas específicas.

El uso de sharding permitiría distribuir la carga de trabajo entre múltiples nodos, mejorar el rendimiento de las consultas y facilitar la escalabilidad horizontal del sistema.

No obstante, debido a que esta propuesta corresponde a una miniaplicación académica, la implementación del sharding no resulta obligatoria en esta etapa, aunque se considera una alternativa viable para futuras expansiones de la plataforma.

6. Queries clave del negocio

A continuación se presentan algunas consultas fundamentales para el funcionamiento de la plataforma ciudadana de denuncias.

➤ **Consulta 1: Buscar denuncias pendientes**

Pregunta de negocio:

¿Cuáles denuncias se encuentran pendientes de atención?

```
db.denuncias.find({  
  estado: "Pendiente"  
})
```

Resultado esperado:

Muestra todas las denuncias que aún no han sido atendidas por la institución responsable.

➤ **Consulta 2: Buscar denuncias por sector**

Pregunta de negocio:

¿Cuántas denuncias existen en un sector específico?

```
db.denuncias.find({  
  sector: "San Isidro"  
})
```

Resultado esperado:

Permite identificar las incidencias reportadas dentro de una zona determinada.

➤ **Consulta 3: Denuncias más recientes**

Pregunta de negocio:

¿Cuáles son las últimas denuncias registradas?

```
db.denuncias.find()  
.sort({  
  fecha_reporte: -1  
})  
.limit(10)
```

Resultado esperado:

Presenta las diez denuncias más recientes registradas en la plataforma.

➤ **Consulta 4: Contar denuncias por estado**

Pregunta de negocio:

¿Cuántas denuncias existen en cada estado?

```
db.denuncias.aggregate([  
  {  
    $group: {  
      _id: "$estado",  
      total: { $sum: 1 }  
    }  
  }  
])
```

Resultado esperado:

Genera un resumen estadístico que facilita la toma de decisiones por parte de los administradores.

➤ **Consulta 5: Denuncias por tipo**

Pregunta de negocio:

¿Cuáles tipos de denuncias reciben más reportes?

```
db.denuncias.aggregate([  
  {  
    $group: {  
      _id: "$tipo_denuncia",  
      cantidad: { $sum: 1 }  
    }  
  }  
])
```

Resultado esperado:

Permite identificar los problemas más frecuentes reportados por los ciudadanos.

7. Arquitectura de la miniaplicación

La solución propuesta está compuesta por tres elementos principales: la interfaz de usuario, el servidor de aplicación y la base de datos MongoDB.

➤ **Componentes**

Frontend

- Interfaz web utilizada por los ciudadanos para registrar y consultar denuncias.
- Permite adjuntar información relacionada con cada incidencia.

Backend

- Desarrollado utilizando Python.
- Gestiona las solicitudes de los usuarios.
- Procesa las consultas hacia MongoDB.
- Actualiza el estado de las denuncias.

Base de datos MongoDB

- Almacena todas las denuncias registradas.
- Gestiona documentos BSON.
- Permite consultas rápidas y escalabilidad futura.

Flujo de funcionamiento

1. El ciudadano registra una denuncia.
2. El backend recibe la información.
3. Los datos se almacenan en MongoDB.
4. Las instituciones consultan y actualizan el estado de la denuncia.
5. El ciudadano puede verificar el progreso de su caso.

Ciudadano



Interfaz Web



Backend (Python)



MongoDB

Esta arquitectura permite una implementación sencilla, escalable y adecuada para los requerimientos de la plataforma ciudadana de denuncias.

8. Implementación

Para la implementación de la solución propuesta se utilizaron herramientas ampliamente empleadas en el desarrollo de aplicaciones basadas en MongoDB.

➤ Herramientas utilizadas

- MongoDB Atlas como servicio de base de datos en la nube.
- MongoDB Compass para la administración visual de la base de datos.
- Visual Studio Code como entorno de desarrollo.
- Python como lenguaje de programación para la lógica de la aplicación.

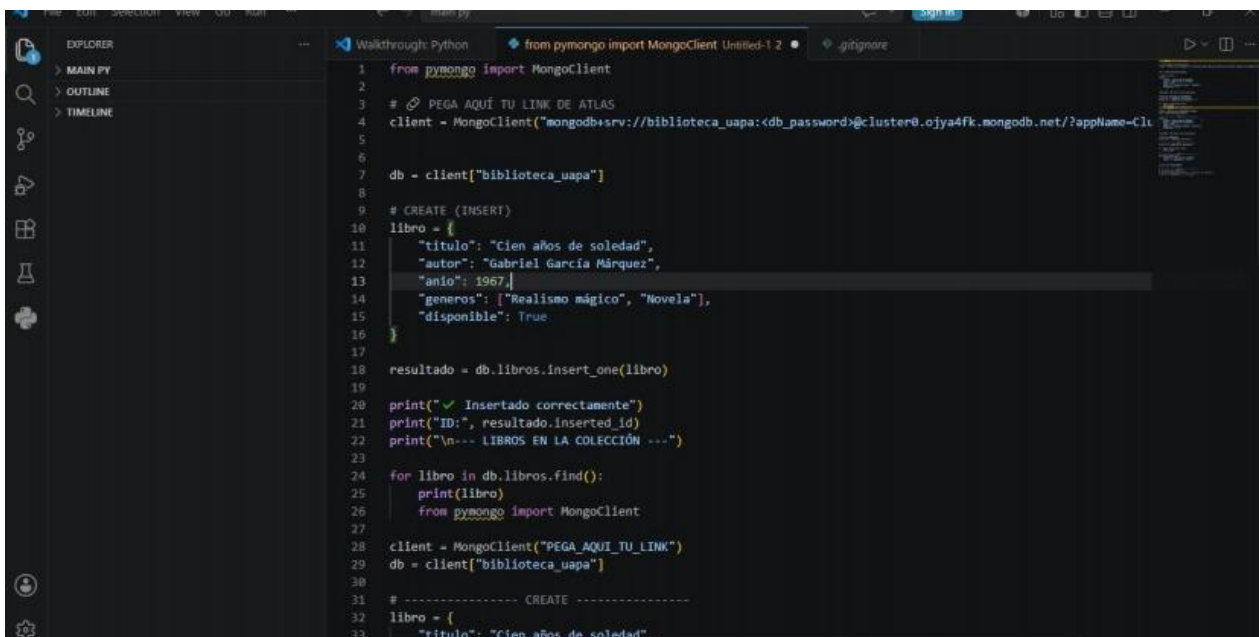
➤ Evidencias de implementación

Durante el desarrollo se realizó la configuración del entorno de trabajo, incluyendo la instalación de MongoDB, la creación de una cuenta en MongoDB Atlas y la conexión con las herramientas necesarias para la administración de la base de datos.

En esta sección se incluyen las capturas de pantalla correspondientes a:

- Configuración de MongoDB Atlas.
- Conexión a la base de datos.
- Entorno de desarrollo en Visual Studio Code.
- Evidencias de las colecciones y documentos almacenados.

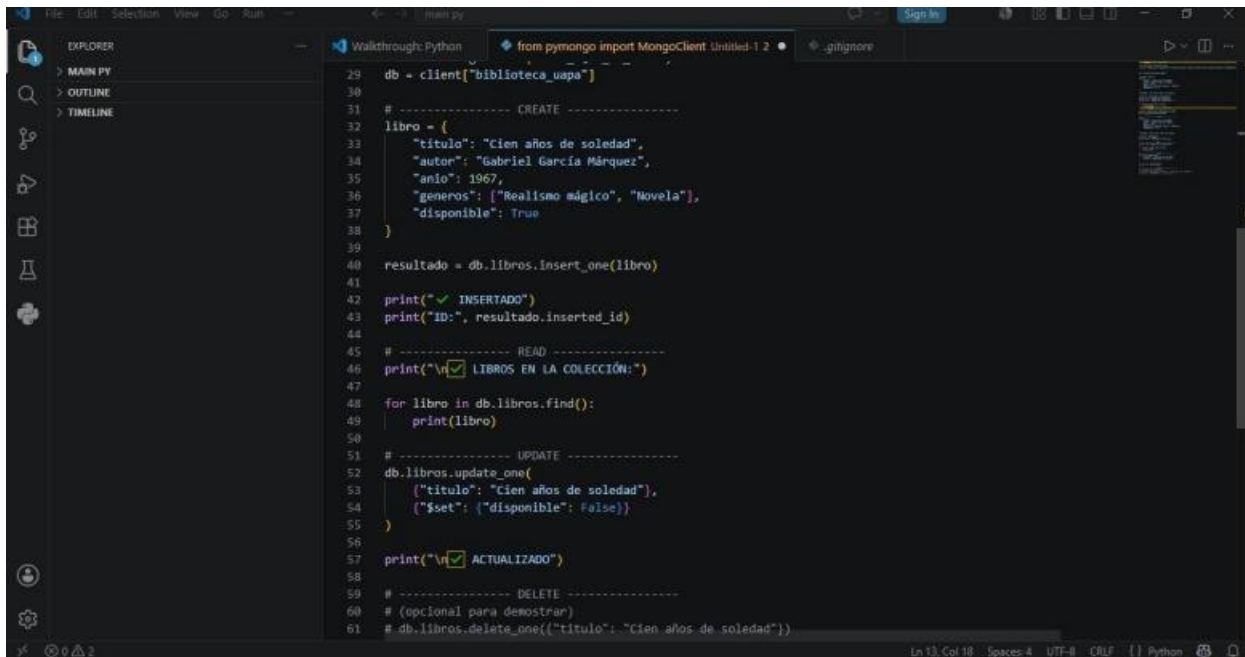
➤ Evidencia de conexión entre Python y MongoDB Atlas.



```
1 from pymongo import MongoClient
2
3 # PEGA AQUÍ TU LINK DE ATLAS
4 client = MongoClient("mongodb+srv://biblioteca_uapa:<db_password>@cluster0.ojya4fk.mongodb.net/?appName=Clu
5
6
7 db = client["biblioteca_uapa"]
8
9 # CREATE (INSERT)
10 libro = {
11     "titulo": "Cien años de soledad",
12     "autor": "Gabriel García Márquez",
13     "año": 1967,
14     "generos": ["Realismo mágico", "Novela"],
15     "disponible": True
16 }
17
18 resultado = db.libros.insert_one(libro)
19
20 print("✓ Insertado correctamente")
21 print("ID:", resultado.inserted_id)
22 print("\n--- LIBROS EN LA COLECCIÓN ---")
23
24 for libro in db.libros.find():
25     print(libro)
26
27 from pymongo import MongoClient
28
29 client = MongoClient("PEGA_AQUI_TU_LINK")
30 db = client["biblioteca_uapa"]
31
32 # ----- CREATE -----
33 libro = {
34     "titulo": "Cien años de soledad",
```

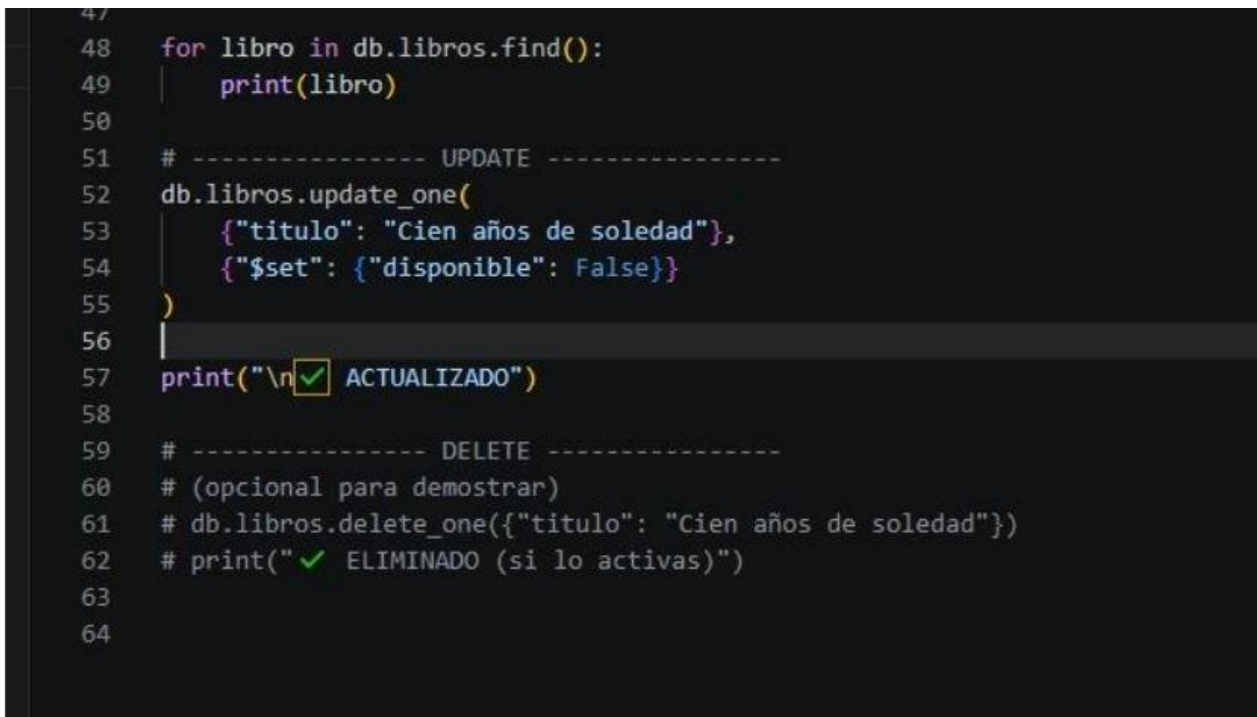
Se realizó la conexión entre Python y MongoDB Atlas mediante la librería PyMongo. Posteriormente se ejecutó una operación de inserción de documentos en la colección correspondiente.

➤ **Evidencia de consulta y actualización de documentos en MongoDB.**

A screenshot of a Python IDE (likely VS Code) with a dark theme. The Explorer sidebar on the left shows a project structure with 'MAIN.PY', 'OUTLINE', and 'TIMELINE'. The main editor window displays a Python script titled 'from pymongo import MongoClient, Untitled-1.2'. The script performs CRUD operations on a MongoDB database named 'biblioteca_uapa'. It creates a document, inserts it, reads all documents, updates one document's 'disponible' status to False, and includes a commented-out delete operation. The script uses green checkmarks in its print statements to indicate successful operations. The status bar at the bottom shows 'Ln 13, Col 10', 'Spaces: 4', 'UTF-8', 'CR/LF', and 'Python'.

Se realizaron consultas utilizando el método find() para recuperar información almacenada en la colección y posteriormente se modificaron documentos mediante el operador \$set.

➤ **Evidencia de modificación de registros mediante operaciones CRUD.**

A close-up screenshot of a Python script showing MongoDB update and delete operations. The code is as follows:

```
47  
48 for libro in db.libros.find():  
49     print(libro)  
50  
51 # ----- UPDATE -----  
52 db.libros.update_one(  
53     {"titulo": "Cien años de soledad"},  
54     {"$set": {"disponible": False}}  
55 )  
56  
57 print("\n✅ ACTUALIZADO")  
58  
59 # ----- DELETE -----  
60 # (opcional para demostrar)  
61 # db.libros.delete_one({"titulo": "Cien años de soledad"})  
62 # print("✅ ELIMINADO (si lo activas)")  
63  
64
```

Se utilizó la operación `update_one()` para actualizar información específica dentro de los documentos almacenados en MongoDB.

➤ **Resultados obtenidos**

La implementación permitió validar la viabilidad del modelo de datos propuesto y comprobar la capacidad de MongoDB para almacenar y consultar información relacionada con las denuncias ciudadanas de manera eficiente y flexible.

9. Instrucciones de reproducción

Para reproducir la solución propuesta se deben seguir los siguientes pasos:

1. Instalar Python 3.x en el equipo.
2. Instalar MongoDB o crear una cuenta en MongoDB Atlas.
3. Instalar Visual Studio Code como entorno de desarrollo.
4. Instalar la librería PyMongo mediante el comando:
`pip install pymongo`
5. Crear la base de datos y la colección correspondiente en MongoDB.
6. Ejecutar los scripts de inserción, consulta y actualización de documentos.
7. Verificar los resultados utilizando MongoDB Compass o MongoDB Atlas.

Siguiendo estos pasos es posible replicar el funcionamiento básico de la solución propuesta y validar las operaciones principales realizadas sobre la base de datos NoSQL.

10. Polyglot Persistence

La solución propuesta utiliza únicamente MongoDB como motor principal de base de datos NoSQL. Por esta razón, no fue necesario implementar una estrategia de polyglot persistence, ya que todos los requerimientos funcionales del sistema pueden ser cubiertos adecuadamente mediante las capacidades ofrecidas por MongoDB.

La elección de un único motor de base de datos permite simplificar la arquitectura de la solución, reducir la complejidad de administración y facilitar el mantenimiento de la aplicación.

No obstante, en escenarios futuros con mayores exigencias tecnológicas, podría evaluarse la incorporación de otros motores especializados para casos específicos, como Redis para almacenamiento en caché o Elasticsearch para búsquedas avanzadas.

11. Lecciones aprendidas y posibles mejoras

Durante el desarrollo de este proyecto fue posible comprender la importancia de las bases de datos NoSQL en escenarios donde la información presenta estructuras flexibles y requiere capacidad de crecimiento. La utilización de MongoDB permitió observar las ventajas del almacenamiento basado en documentos y la facilidad para gestionar información semiestructurada.

Asimismo, se fortalecieron conocimientos relacionados con el diseño de colecciones, tipos BSON, creación de índices y ejecución de consultas orientadas a las necesidades del negocio. También se evidenció la importancia de seleccionar adecuadamente el motor de base de datos según los patrones de acceso y los requerimientos de la aplicación.

Entre las principales mejoras que podrían implementarse en futuras versiones del sistema se encuentran:

- Incorporación de autenticación y control de acceso para usuarios y administradores.
- Integración de notificaciones automáticas para informar el estado de las denuncias.
- Implementación de geolocalización para identificar con mayor precisión la ubicación de las incidencias.
- Desarrollo de paneles estadísticos avanzados para apoyar la toma de decisiones institucionales.
- Implementación de aplicaciones móviles para Android e iOS.

Estas mejoras permitirían incrementar la funcionalidad de la plataforma y ofrecer una experiencia más completa tanto para los ciudadanos como para las instituciones responsables de gestionar las denuncias

12. Conclusión

El desarrollo de este proyecto permitió aplicar los conocimientos adquiridos sobre bases de datos NoSQL en la solución de un problema real relacionado con la gestión de denuncias ciudadanas. A través del análisis de los requerimientos del negocio se diseñó una propuesta basada en MongoDB, aprovechando las ventajas que ofrece este motor para el manejo de información flexible y en constante crecimiento.

La solución propuesta facilita el registro, almacenamiento y seguimiento de denuncias realizadas por los ciudadanos, contribuyendo a una gestión más eficiente por parte de las instituciones responsables. Asimismo, el diseño realizado demuestra la capacidad de MongoDB para manejar documentos semiestructurados, consultas eficientes y escalabilidad futura.

Durante el proyecto se analizaron los patrones de acceso, se definieron estructuras de datos adecuadas, se propusieron índices para optimizar las consultas y se identificaron posibles mejoras para futuras versiones de la plataforma. Estos elementos permitieron comprender la importancia de seleccionar correctamente la tecnología de almacenamiento según las necesidades específicas de cada organización.

Finalmente, se concluye que MongoDB representa una alternativa adecuada para este tipo de solución debido a su flexibilidad, facilidad de implementación y capacidad para adaptarse a escenarios con grandes volúmenes de información. La experiencia obtenida fortalece las competencias necesarias para diseñar e implementar soluciones modernas basadas en tecnologías NoSQL dentro del ámbito profesional.

REFERENCIAS BIBLIOGRÁFICAS

- [MongoDB, Inc. \(2025\). MongoDB Documentation. https://www.mongodb.com/docs/](https://www.mongodb.com/docs/)
- [MongoDB, Inc. \(2025\). MongoDB Atlas Documentation. https://www.mongodb.com/atlas/database](https://www.mongodb.com/atlas/database)
- [Chodorow, K. \(2020\). MongoDB: The Definitive Guide \(4th ed.\). O'Reilly Media.](#)
- [Fowler, M., & Sadalage, P. J. \(2013\). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.](#)
- [Sarasa, A. \(2021\). Bases de datos NoSQL y Big Data. eLibro/UAPA.](#)
- [Tejada Betancourt, R. \(2022\). Gestión de datos para inteligencia de negocios. eLibro/UAPA.](#)
- [Oracle Corporation. \(2024\). Database Concepts. https://docs.oracle.com](https://docs.oracle.com)
- [IBM Corporation. \(2024\). Introduction to NoSQL Databases. https://www.ibm.com](https://www.ibm.com)